Real-Time American Sign Language (ASL) Recognition and Bidirectional Learning System

A Project Report submitted in partial fulfillment of the requirements for the completion of a guided project

Submitted by

Raghuram Duvvuri

Harshita Chemboli

Under the esteemed guidance of
Dr. Soujanya Duvvi
Assistant Professor

GITAM
DEEMED TO BE UNIVERSITY

Department of Computer Science and Engineering
GITAM School of Computer Science and Engineering
GITAM (Deemed to be University)
Visakhapatnam

2026

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to **Dr. Soujanya Duvvi, Assistant Professor, Department of Computer Science and Engineering, GITAM School of Technology, GITAM (Deemed to be University)**, Visakhapatnam, for her invaluable guidance, constant support, and encouragement throughout the course of our project. Her insights and mentorship have been instrumental in enhancing our learning experience and helping us understand this work successfully.

We are also deeply thankful to **GITAM (Deemed to be University)** for providing us with this valuable opportunity that enabled the successful completion of this project.

TABLE OF CONTENTS

1. ABSTRACT

American Sign Language (ASL) serves as the primary mode of communication for millions of people in the deaf community, yet the barrier between ASL users and people who do not know it remains a significant challenge in everyday interactions. This project addresses the communication gap with a real-time, bidirectional ASL recognition system that uses computer vision and deep learning for seamless translation between sign language and text. The system utilizes MediaPipe for hand-landmark detection through live input, feeding gesture data into a custom-trained PyTorch neural network for ASL gesture classification. A Streamlit interface provides easy usage, making communication accessible in both directions. The result is an interactive application that demonstrates the power of AI-driven assistive technology in bridging the communication divide between ASL users and people who do not know it.

2. INTRODUCTION

Communication is required for a stable life, yet for the approximately 70 million deaf and hard-of-hearing individuals worldwide, everyday interaction with the common population presents a challenge. American Sign Language, a fully expressive visual-gestural language, serves as the primary method of communication for a significant portion of this community. Despite its widespread use, the vast majority of hearing individuals lack knowledge of ASL, creating a communication barrier that affects access to education, healthcare, employment, and social inclusion.

Traditional solutions to this problem have been largely manual and impractical at scale. Human interpreters are expensive, not universally available, and impossible to deploy in spontaneous or private interactions. Early assistive technologies relied on specialized glove-based hardware equipped with sensors to detect hand movements, making them costly and difficult to adopt in many settings. These limitations highlighted a clear and urgent need for an accessible and intelligent system capable of bridging the gap between ASL users and the common public.

Recent advancements in computer vision, machine learning, and real-time image processing have opened new possibilities for gesture recognition without the need for any specialized hardware. Frameworks such as MediaPipe now enable highly accurate hand-landmark detection using nothing more than a standard webcam, while deep learning architectures built on platforms like PyTorch allow for robust classification of complex gestures with high accuracy and low latency. Together, these technologies lay the groundwork for a software-driven ASL recognition system.

Existing systems suffer from notable shortcomings. Most are unidirectional, meaning they translate in only one direction. Furthermore, very few systems offer an integrated interface accessible to both signers and non-signers simultaneously.

This project is titled Real-Time American Sign Language (ASL) Recognition and Bidirectional Learning System. The title shows the main functions of this system: real-time processing capability, accurate ASL gesture recognition through deep learning, and a bidirectional interface that supports both gesture-to-text translation and text-to-gesture generation. By combining these elements into a single, accessible application built on Streamlit, the system provides an easy way for both ASL users and the common public to communicate seamlessly.

3. PROBLEM IDENTIFICATION

Despite the widespread use of American Sign Language among the deaf and hard-of-hearing community, no scalable and affordable solution currently exists that enables fluid, real-time communication between ASL users and the general hearing population. Existing systems are either hardware-dependent, computationally expensive, unidirectional, or limited to recognizing only a narrow set of static gestures under controlled conditions. There is a clear need for an intelligent, camera-based system that can accurately interpret ASL gestures in real time and make communication accessible to both sides of the conversation, without requiring any specialized equipment or prior knowledge of sign language from the user.

- To implement real-time hand detection using MediaPipe to accurately capture and track hand landmarks from live webcam input without the need for any external hardware.
- To design and train a deep learning model using the PyTorch framework capable of classifying a defined set of ASL gestures with high accuracy and minimal latency.
- To develop a bidirectional communication interface using Streamlit that supports both gesture-to-text translation for ASL users and text-to-gesture output for hearing individuals learning or responding in sign language.
- To contribute a scalable and extensible foundation that can be further developed to support dynamic gestures, expanded ASL vocabulary, and multilingual sign language systems in future iterations.

4. EXISTING SYSTEMS

4.1 HARDWARE-BASED GESTURE RECOGNITION (GLOVE SYSTEMS)

One of the earliest approaches to sign language recognition involved the use of instrumented gloves embedded with flex sensors, accelerometers, and gyroscopes to capture hand movements. While these devices could capture precise finger and wrist movements, they were expensive and uncomfortable for daily use. The requirement for specialized hardware made the technology inaccessible to the general public and impractical for spontaneous communication scenarios. Additionally, these systems required individual calibration for each user and offered limited portability, making them unsuitable for seamless, everyday communication.

4.2 UNIDIRECTIONAL AND STATIC RECOGNITION SYSTEMS

A major limitation across all existing systems is their unidirectional nature. Almost every prior solution was designed exclusively to translate sign gestures into text, with no reverse functionality allowing hearing users to communicate back in sign language. This one-sided design made existing tools useful only as translation aids rather than true communication bridges. Furthermore, the vast majority of systems were restricted to recognizing static, isolated hand postures, failing to account for the dynamic, continuous motion that characterizes natural ASL communication. The absence of a user-friendly interface in most implementations also meant that these tools remained confined to research environments, with little practical accessibility for everyday users or those looking to learn sign language.

5. PROPOSED SYSTEM

The proposed system is a real-time, bidirectional American Sign Language recognition application that addresses the shortcomings of existing approaches through an entirely software-driven architecture. By combining computer vision, deep learning, and an intuitive user interface, the system eliminates the need for any specialized hardware while delivering accurate, low-latency gesture recognition accessible to both ASL users and the hearing population. The integration of these technologies into a single, unified application represents a significant advancement over the fragmented, unidirectional, and hardware-dependent solutions that already exist.

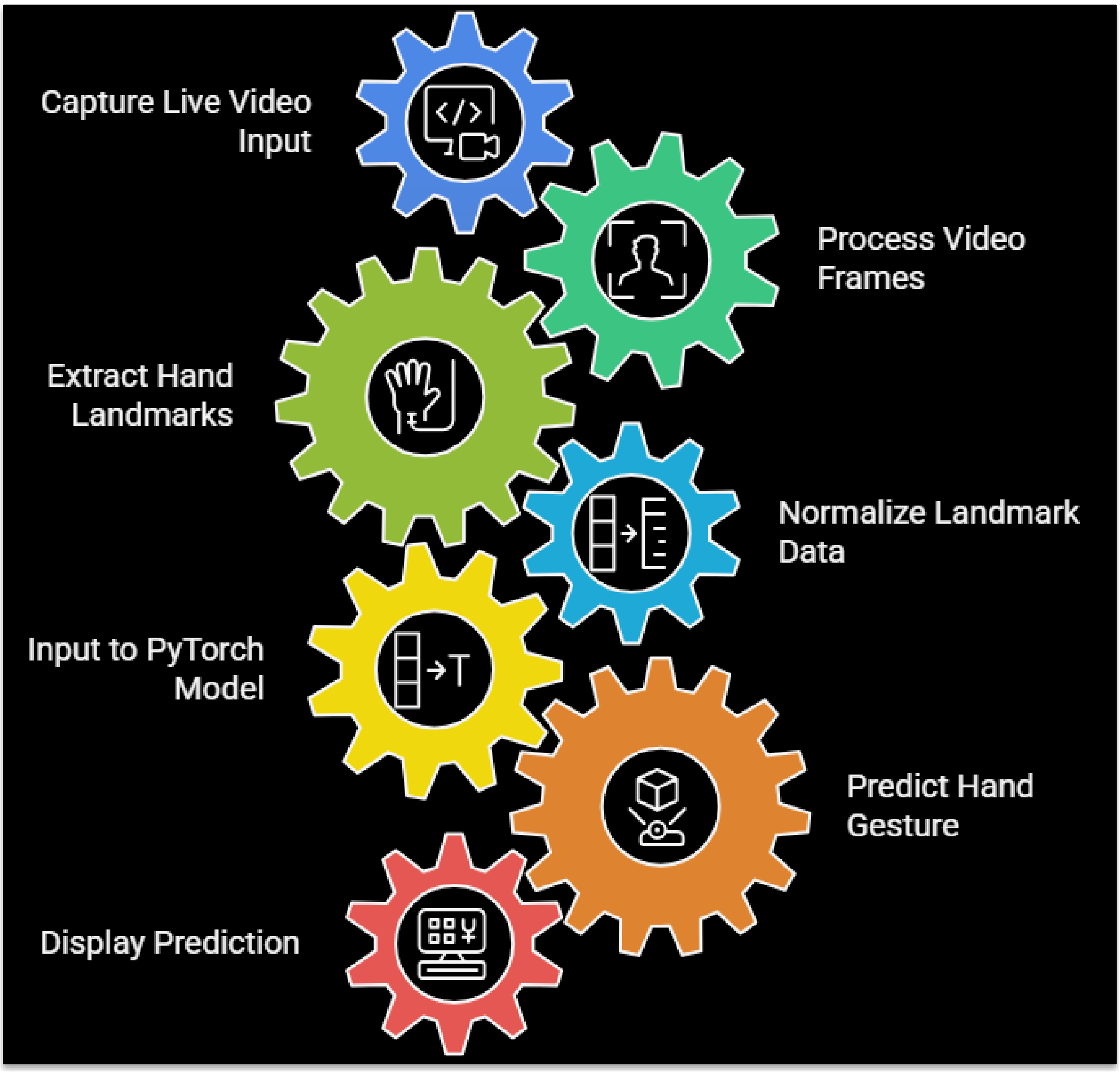

Fig 1: Project Workflow Diagram

5.1 HAND DETECTION & LANDMARK EXTRACTION USING MEDIAPIPE

At the core of the system's input system is Google's MediaPipe framework, which is responsible for real-time hand detection and landmark tracking through a standard webcam. MediaPipe identifies and tracks 21 precise key points on each hand, generating a spatial representation of hand position and posture with each video frame. This approach operates effectively across a wide range of lighting conditions, skin tones, and hand sizes. By replacing the hardware of earlier glove-based systems, MediaPipe provides a robust and universally accessible foundation for gesture capture that previous systems could not offer.

5.2 DEEP LEARNING CLASSIFICATION USING PYTORCH

The spatial landmark data extracted by MediaPipe is fed into a custom-designed neural network built and trained using the PyTorch deep learning framework. The neural network learns to identify meaningful patterns directly from the landmark coordinates, eliminating the need for any hand-crafted feature extraction steps. The model is trained on a dataset covering the full ASL alphabet and a defined vocabulary of gestures, with sufficient data diversity to ensure strong generalization across different users and environments. The architecture is optimized for both classification accuracy and inference speed, ensuring that gesture predictions are delivered in real time with minimal latency. This represents a direct and measurable improvement over prior systems that sacrificed accuracy for speed.

5.3 BIDIRECTIONAL USER INTERFACE USING STREAMLIT

One of the most significant innovations of the proposed system over all existing solutions is its bidirectional communication capability through a Streamlit-based web interface. The system operates in two distinct modes. In the first mode, gesture-to-text translation, the application captures live hand gestures through the webcam, processes them through the MediaPipe and PyTorch system, and displays the recognized ASL characters or words as text in real time. This allows deaf and hard-of-hearing individuals to communicate naturally with hearing users who have no knowledge of sign language. In the second mode, text-to-gesture generation, hearing users can input text through the interface and receive a corresponding visual representation of the associated ASL gesture, enabling them to respond in sign language. This dual functionality transforms the system from a simple translation tool into a comprehensive communication and educational platform.

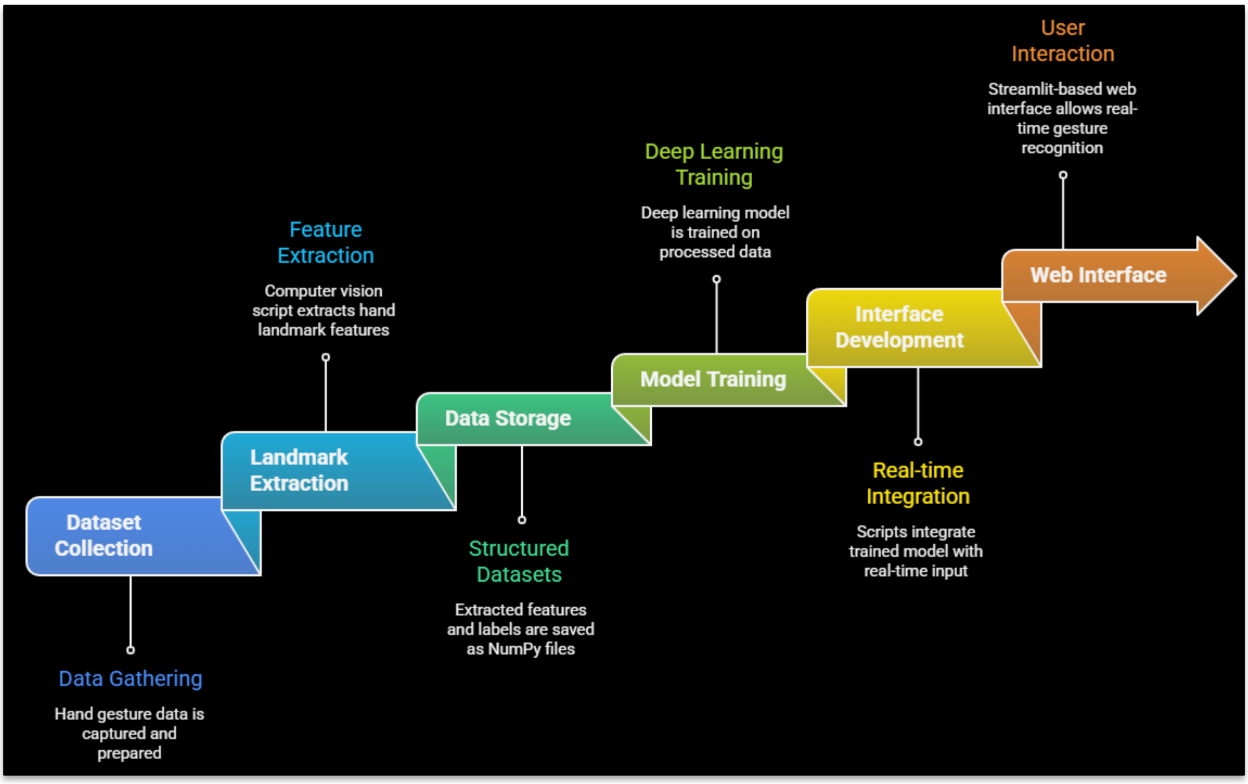

Fig 2: End-to-End Project Pipeline Diagram

6. SYSTEM ARCHITECTURE

The input layer is the entry point of the system. The user performs an ASL gesture in front of a webcam. The live video feed is captured frame by frame and passed into the processing pipeline. MediaPipe receives each video frame and performs real-time hand detection. It identifies the hand within the frame and extracts 21 precise key-point coordinates corresponding to the wrist, knuckle joints, and fingertip positions of each finger. These coordinates form a compact but highly informative spatial representation of the gesture, which is then forwarded to the classification model. A PyTorch CNN is the intelligence core of the system. The 21 landmark coordinates are fed into a custom-trained CNN that has learned to associate specific patterns with specific ASL gestures. The model outputs a class prediction identifying which letter or word from the supported ASL vocabulary the gesture corresponds to, with high accuracy and minimal latency. A Streamlit Web Interface acts as the routing layer of the application. Once a prediction is received, the interface determines which of the two operating modes is active and routes the output accordingly. This layer also handles all user interaction, including mode switching, text input, and display rendering. The output layer is two distinct communication pathways the system supports. In gesture-to-text mode, the recognized ASL gesture is rendered as readable text on screen, allowing hearing users to understand what the signer is communicating. In text-to-gesture mode, a hearing user's typed input is mapped to a visual representation of the corresponding ASL gesture, enabling them to respond in sign language. Together, these two output paths complete the bidirectional communication loop that distinguishes this system from all prior systems.

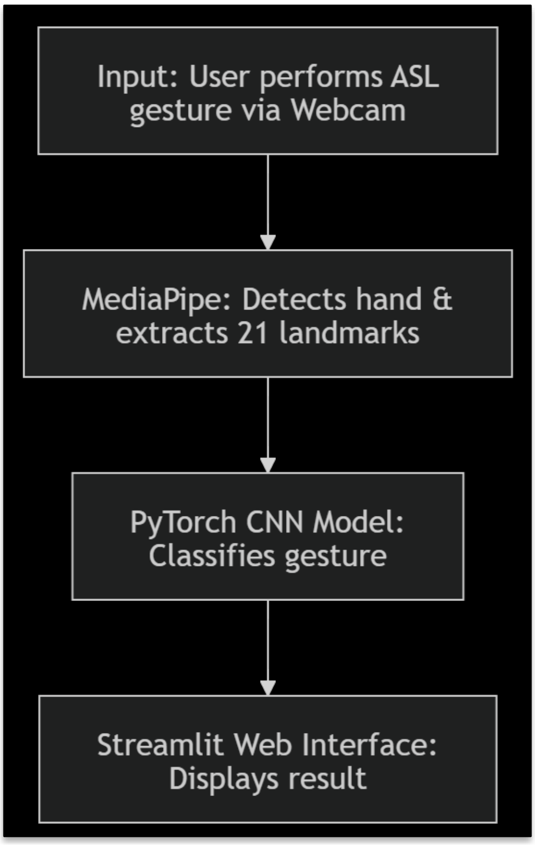

Fig 3: System Architecture Diagram

7. TOOLS/TECHNOLOGIES USED

Python

Python served as the primary programming language for the entire project. Its extensive ecosystem of machine learning libraries, combined with its clean and readable syntax, made it the natural choice for a project that spans computer vision, neural network training, and web interface development. Python's versatility allowed all components of the system, from data preprocessing and model training to real-time inference and UI rendering, to be developed and integrated within a single, unified codebase, significantly reducing development complexity and time.

MediaPipe

MediaPipe is an open-source framework for building real-time perception systems. In this project, MediaPipe's Hand Landmarker was used to detect the user's hand within each live video frame and extract 21 three-dimensional key-point coordinates corresponding to the wrist, finger joints, and fingertips. This landmark data provides an expressive representation of hand posture that is both lighting-invariant and skin-tone agnostic, overcoming one of the most persistent weaknesses of classical image processing approaches. By replacing pixel-level feature extraction with structured geometric data, MediaPipe simplifies the classification task and improves robustness across real-world conditions.

PyTorch

PyTorch is an open-source deep learning framework, widely used in both academic research and industry for its flexibility and dynamic computation graph. In this project, PyTorch was used to design, train, and deploy the gesture classification neural network. The framework's intuitive API allowed for rapid prototyping of the network architecture, while its support for GPU-accelerated training enabled efficient model optimization over the full gesture dataset. At inference time, PyTorch's lightweight deployment capabilities ensured that the trained model could deliver real-time predictions with minimal latency, making it well-suited for integration into a live webcam-based application.

Streamlit

Streamlit is an open-source Python framework designed for rapidly building interactive web applications. In this project, Streamlit was used to develop the bidirectional user interface that ties the entire system together. It enabled the creation of a clean, browser-accessible application without requiring complex solutions to integrate the Python-based into a web app. Streamlit's support for real-time data rendering made it possible to display gesture recognition results as they are produced, while its input widgets helped the text-to-gesture communication mode for hearing users.

OpenCV .

OpenCV is one of the most widely used libraries for real-time image and video processing. In this project, OpenCV was used to interact with the webcam, capture live video frames, and preprocess each frame before passing it to the MediaPipe pipeline. It handled tasks such as frame resizing, color space conversion, and rendering visual overlays on the video feed to display detected landmarks and recognition results. OpenCV's efficiency and compatibility with Python made it an essential utility layer bridging the raw video input and the intelligent processing pipeline.

NumPy

NumPy is a numerical computing library for Python, providing support for large multi-dimensional arrays and a wide range of mathematical operations. In this project, NumPy was used for processing and transforming the landmark coordinate data extracted by MediaPipe into the structured numerical arrays expected by the PyTorch model. Its efficiency ensured that these operations did not introduce any delay into the real-time processing system.

8. IMPLEMENTATION

8.1 LANDMARK EXTRACTION AND NORMALIZATION

The extract_lm.py file processes the raw ASL image dataset and converts it into structured numerical data suitable for model training. For each image, MediaPipe's static image mode detects the hand and returns 21 three-dimensional landmark coordinates. These coordinates are normalized by first subtracting the wrist landmark so that the representation is position-independent, then dividing by the maximum absolute coordinate value so that all values fall within a consistent range. This normalization ensures the model learns the shape of each gesture rather than where the hand appears in the frame. The resulting 63-element feature vectors and their class labels are saved as NumPy arrays.

8.2 MODEL ARCHITECTURE

The ASLModel class defines a fully connected feedforward neural network with four linear layers of widths 63, 256, 128, 64, and 26. Batch normalization and dropout with a rate of 0.3 are applied after the first two hidden layers to prevent overfitting, and ReLU activations are used throughout. The output layer produces 26 logits, one for each letter of the ASL alphabet.

8.3 MODEL TRAINING

The train_model.py file loads the saved landmark arrays, wraps them in a custom Dataset class, and splits them 80/20 into training and test sets. The model is trained for 30 epochs using the Adam optimizer with a learning rate of 0.001 and cross-entropy loss. After each epoch, training accuracy, test accuracy, and time elapsed are printed to monitor progress. The trained model's state dictionary is saved to disk upon completion.

8.4 LIVE INFERENCE

The live_model.py file opens the webcam and processes each frame in real time. Each frame is flipped horizontally, passed through MediaPipe to extract hand landmarks, normalized using the same procedure applied during dataset preparation, and fed into the trained model. Softmax is applied to the output logits, and the predicted letter along with its confidence score is overlaid on the video frame using OpenCV.

8.5 STREAMLIT WEB INTERFACE

The web_integration.py file implements the bidirectional interface as a two-tab Streamlit application. The first tab allows the user to select a letter from a dropdown and displays the corresponding ASL reference image. The second tab activates the webcam and runs the full inference pipeline, rendering the annotated video feed and the predicted letter live in the browser. The model and MediaPipe hands object are loaded once at startup using Streamlit's cache_resource decorator to avoid redundant reloading across reruns.

9. RESULTS

9.1 LETTER TO GESTURE

The first tab of the Streamlit interface allows a user to select any letter from a dropdown and immediately view the corresponding ASL hand sign. The reference images loaded correctly for all 26 letters, providing an accessible visual guide for hearing users looking to respond in sign language.

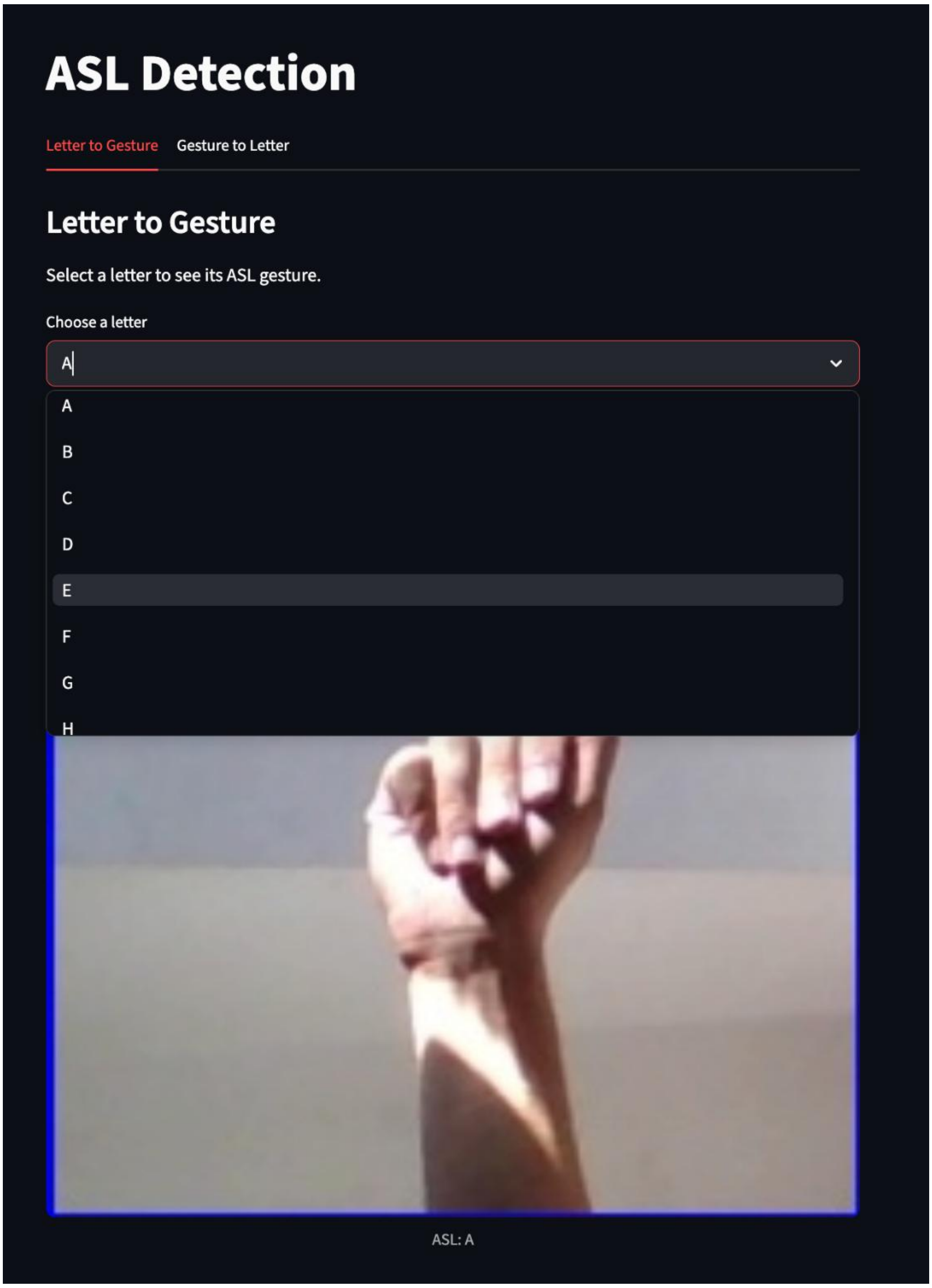

Fig 4: Letter to Gesture mapping

9.2 GESTURE TO LETTER

The second tab demonstrates the core gesture recognition capability of the system. With the webcam active, the application successfully detected and classified ASL hand signs in real time, displaying the predicted letter and confidence score overlaid on the live video feed. Predictions were produced with high confidence for clear, well-formed gestures, and the MediaPipe hand skeleton overlay confirmed accurate landmark tracking across different hand positions and lighting conditions.

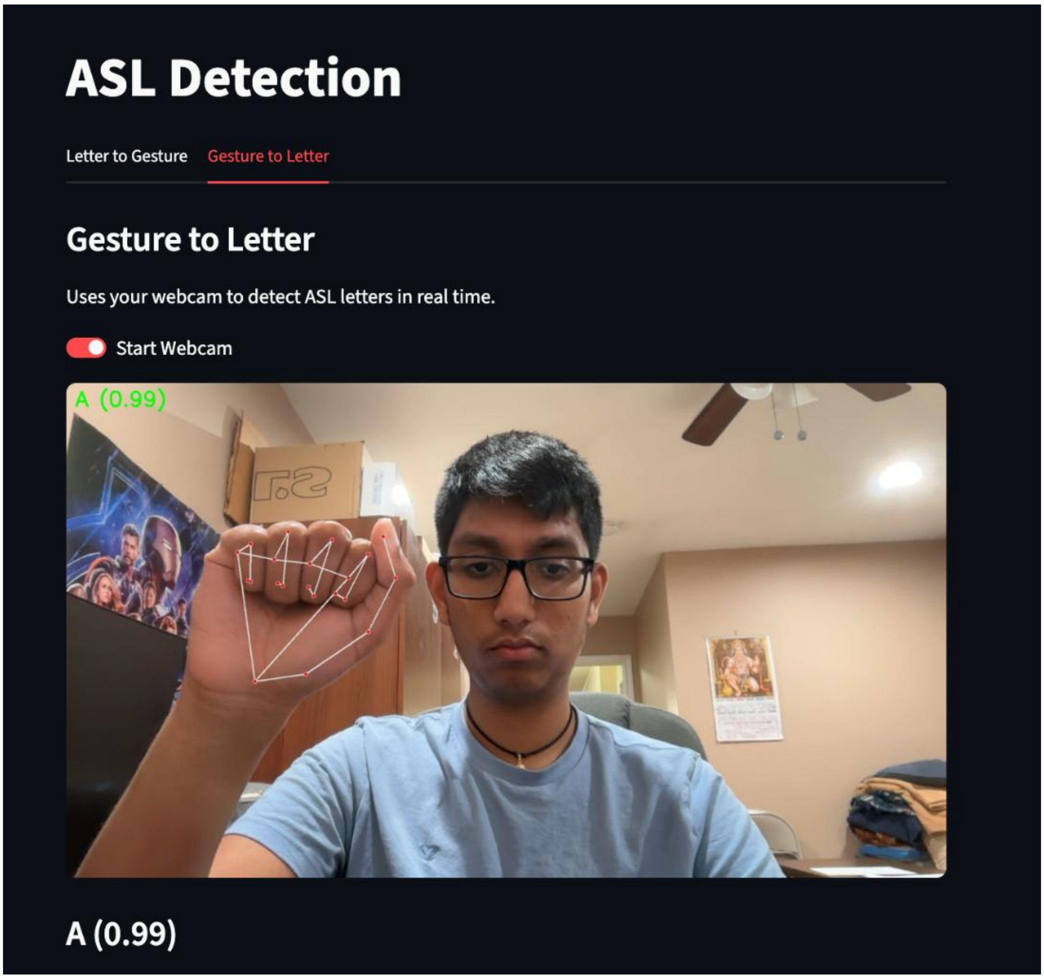

Fig 5: Gesture Recognition to Letter

10.CONCLUSION

The Real-Time American Sign Language Recognition and Bidirectional Learning System represents a meaningful step forward in the effort to bridge the communication gap between the deaf and hard-of-hearing community and the broader hearing population. By integrating Google's MediaPipe framework for precise hand landmark detection, a custom-trained PyTorch neural network for accurate gesture classification, and a Streamlit-based interface for bidirectional communication, the project delivers a complete, software-driven solution that is accessible, practical, and scalable — without reliance on any specialized hardware.

PROJECT PROFORMA

Classification of the Project	Application	Product	Research	Review
	✓			

Project Outcomes	
Outcome 1	Formulate specific problem statements for well-defined real-life problems with reasonable assumptions and constraints.
Outcome 2	Perform literature search and / or patent search in the area of interest.
Outcome 3	Conduct experiments / Design and Analysis / solution iterations and document the results.
Outcome 4	Perform error analysis / benchmarking / costing, Synthesis the results and arrive at scientific conclusions / products / solution
Outcome 5	Document the results in the form of technical report / presentation

GITAM
DEEMED TO BE UNIVERSITY

Bengaluru

NH 207, Nagadenehali,
Doddaballapur Taluk,
Bengaluru - 561203,
Karnataka, India

Hyderabad

Rudraram, Patancheru
mandal, Hyderabad -
502329, Telengana, India

Visakhapatnam

Gandhi Nagar, Rushikonda,
Visakhapatnam- 530045,
Andhra Pradesh, India

GIMSR
Medical College Rd,
Rushikonda,

Visakhapatnam - 530045
Pradesh, India

Bengaluru City Office
No 5/1, First Floor, Prestige
Terraces, Union Street,

Infantry Road, Bengaluru -
560003 Karnataka, India

Hyderabad City Office GITAM
6th floor, TP House, Cyber Gandhi Nagar,
Hills Colony, VIP Hills,

Jaihind Enclave, Madhapur, Andhra
Hyderabad - 500081,Telangana, India


Bengaluru


Visakhapatnam


Hyderabad


GIMSR